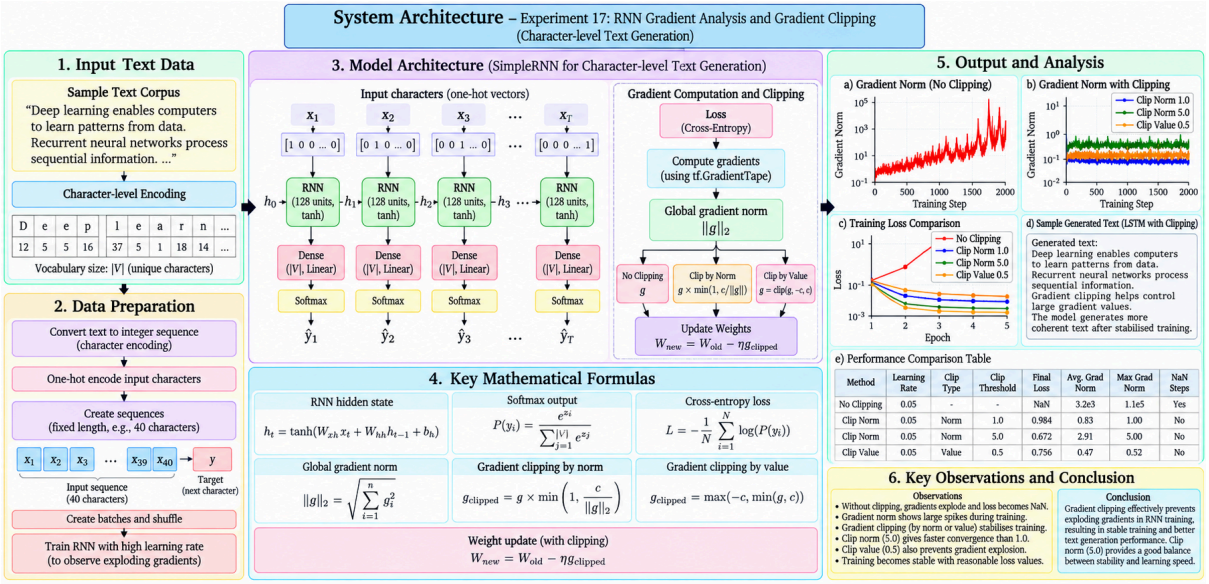
Experiment 17: RNN Gradient Analysis and Gradient Clipping

1. Aim

To implement a character-level Recurrent Neural Network (RNN) for text generation and analyse the problem of exploding gradients during training. To calculate and monitor global gradient norms using TensorFlow's `GradientTape`, identify gradient spikes and NaN losses, apply gradient clipping using different clipping thresholds, compare training loss and gradient norm curves, and determine a suitable clipping threshold for stable RNN training.



2. Mathematical Formulas

1. RNN Hidden State

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

2. Output Probability (Softmax)

$$P(y_i) = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}$$

3. Cross-Entropy Loss

$$L = -\frac{1}{N}\sum_{i=1}^{N} \log(P(y_i))$$

4. Global Gradient Norm

$$\|g\|_2 = \sqrt{\sum_{i=1}^{n} g_i^2}$$

Where g_i represents each gradient element across the trainable parameters.

5. Gradient Clipping by Norm

$$g_{\text{clipped}} = g \times \min\left(1, \frac{c}{\|g\|_2}\right)$$

Where c is the clipping threshold.

6. Gradient Clipping by Value

$$g_{\text{clipped}} = \max(-c, \min(g, c))$$

Each gradient element is restricted to the range $[-c, c]$.

7. Weight Update

$$W_{\text{new}} = W_{\text{old}} - \eta g_{\text{clipped}}$$

Where η represents the learning rate.

8. Exploding Gradient Condition

$$\|g\|_2 \gg 1$$

Very large gradients can produce unstable weight updates and may lead to NaN losses.

9. Training Loss Comparison

$$L_{\text{avg}} = \frac{1}{E}\sum_{i=1}^{E} L_i$$

CODE

```
# Experiment 17: RNN Gradient Analysis and Gradient Clipping

import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
import pandas as pd

np.random.seed(42)
tf.random.set_seed(42)

# ========================================
# 1. CREATE CHARACTER-LEVEL TEXT DATASET
# ========================================

text = """
Deep learning enables computers to learn patterns from data.
Recurrent neural networks process sequential information.
Gradient descent updates weights to minimize prediction loss.
Exploding gradients make neural network training unstable.
Gradient clipping helps control large gradient values.
Artificial intelligence is used in many real world applications.
""" * 100

# Character vocabulary
vocab = sorted(set(text))

char_to_idx = {ch: i for i, ch in enumerate(vocab)}
idx_to_char = {i: ch for ch, i in char_to_idx.items()}

vocab_size = len(vocab)

encoded = np.array(
    [char_to_idx[ch] for ch in text],
    dtype=np.int32
)

print("Total Characters:", len(encoded))
print("Vocabulary Size:", vocab_size)

# ========================================
# 2. CREATE SEQUENCES
# ========================================

sequence_length = 40

X = []
y = []

for i in range(len(encoded) - sequence_length):
    X.append(encoded[i:i + sequence_length])
    y.append(encoded[i + 1:i + sequence_length + 1])

X = np.array(X)
y = np.array(y)

# One-hot encoding
X_onehot = tf.one_hot(X, depth=vocab_size)

dataset = tf.data.Dataset.from_tensor_slices(
```

```
    (X_onehot, y)
)

dataset = dataset.batch(64).prefetch(
    tf.data.AUTOTUNE
)

print("Input Shape:", X_onehot.shape)
print("Target Shape:", y.shape)

# ============================================
# 3. BUILD CHARACTER-LEVEL RNN
# ============================================

def build_rnn():

    model = tf.keras.Sequential([
        tf.keras.layers.Input(
            shape=(sequence_length, vocab_size)
        ),

        tf.keras.layers.SimpleRNN(
            128,
            activation="tanh",
            return_sequences=True,
            recurrent_initializer="glorot_uniform"
        ),

        tf.keras.layers.Dense(vocab_size)
    ])

    return model

# ============================================
# 4. TRAIN WITH GRADIENT MONITORING
# ============================================

def train_model(name, clipping=None, threshold=None):

    tf.keras.backend.clear_session()
    tf.random.set_seed(42)

    model = build_rnn()

    # Deliberately high learning rate
    optimizer = tf.keras.optimizers.SGD(
        learning_rate=0.05
    )

    loss_function = tf.keras.losses.SparseCategoricalCrossentropy(
        from_logits=True
    )

    gradient_history = []
    loss_history = []
    nan_count = 0

    print("\nTraining:", name)

    for epoch in range(5):
```

```
        batch_losses = []

        for batch_x, batch_y in dataset:

            with tf.GradientTape() as tape:

                predictions = model(
                    batch_x, training=True
                )

                loss = loss_function(
                    batch_y, predictions
                )

            gradients = tape.gradient(
                loss, model.trainable_variables
            )

            # Calculate original global gradient norm
            global_norm = tf.linalg.global_norm(gradients)

            norm_value = float(global_norm.numpy())

            gradient_history.append(norm_value)

            # Detect NaN or infinite gradients/loss
            finite = (
                bool(tf.math.is_finite(loss).numpy())
                and bool(tf.math.is_finite(global_norm).numpy())
                and all(
                    bool(tf.reduce_all(tf.math.is_finite(g)).numpy())
                    for g in gradients if g is not None
                )
            )

            if not finite:
                nan_count += 1
                continue

            # Apply gradient clipping
            if clipping == "norm":

                gradients, _ = tf.clip_by_global_norm(
                    gradients, threshold
                )

            elif clipping == "value":

                gradients = [
                    tf.clip_by_value(
                        g, -threshold, threshold
                    ) if g is not None else None
                    for g in gradients
                ]

            # Update model weights
            optimizer.apply_gradients(
                zip(gradients, model.trainable_variables)
            )

            batch_losses.append(float(loss.numpy()))
```

```
        epoch_loss = (
            np.mean(batch_losses)
            if batch_losses else np.nan
        )

        loss_history.append(epoch_loss)

        print(
            f"Epoch {epoch+1}/5 - "
            f"Loss: {epoch_loss:.4f}"
        )

    return {
        "name": name,
        "gradients": gradient_history,
        "loss": loss_history,
        "nan_count": nan_count,
        "model": model
    }

# ========================================
# 5. COMPARE CLIPPING METHODS
# ========================================

experiments = [
    ("No Clipping", None, None),
    ("Clip Norm 1.0", "norm", 1.0),
    ("Clip Norm 5.0", "norm", 5.0),
    ("Clip Value 0.5", "value", 0.5)
]

results = {}

for name, method, threshold in experiments:

    results[name] = train_model(
        name,
        clipping=method,
        threshold=threshold
    )

# ========================================
# 6. DISPLAY GRADIENT NORM COMPARISON
# ========================================

plt.figure(figsize=(12, 6))

for name, result in results.items():

    plt.plot(
        result["gradients"],
        label=name,
        alpha=0.8
    )

plt.xlabel("Training Step")
plt.ylabel("Global Gradient Norm")
plt.title("Gradient Norm Comparison")
plt.legend()
plt.grid(alpha=0.3)
```

```
plt.show()

# ========================================
# 7. DISPLAY LOSS CURVES
# ========================================

plt.figure(figsize=(10, 5))

for name, result in results.items():

    plt.plot(
        range(1, 6),
        result["loss"],
        marker="o",
        label=name
    )

plt.xlabel("Epoch")
plt.ylabel("Training Loss")
plt.title("Training Loss Comparison")
plt.legend()
plt.grid(alpha=0.3)
plt.show()

# ========================================
# 8. IDENTIFY GRADIENT SPIKES
# ========================================

spike_threshold = 5.0

for name, result in results.items():

    gradients = np.array(result["gradients"])

    spikes = np.where(
        np.isfinite(gradients) &
        (gradients > spike_threshold)
    )[0]

    print("\n", name)
    print("Gradient Spikes:", len(spikes))
    print("Non-finite Loss/Gradient Batches:",
          result["nan_count"])

    if len(spikes) > 0:
        print("Spike Steps:", spikes[:10])

# ========================================
# 9. CREATE PERFORMANCE TABLE
# ========================================

summary = []

for name, result in results.items():

    gradients = np.array(result["gradients"])
    losses = np.array(result["loss"])

    summary.append({
        "Method": name,
        "Average Gradient Norm": np.nanmean(gradients),
```

```
        "Maximum Gradient Norm": np.nanmax(gradients),
        "Final Training Loss": losses[-1],
        "Non-finite Batches": result["nan_count"]
    })

summary_df = pd.DataFrame(summary)

print("\nGRADIENT CLIPPING COMPARISON")
print(summary_df.round(4).to_string(index=False))

# ==========================================
# 10. RECOMMEND CLIPPING THRESHOLD
# ==========================================

valid_results = summary_df[
    np.isfinite(summary_df["Final Training Loss"])
]

if len(valid_results) > 0:

    recommended = valid_results[
        valid_results["Method"].str.startswith("Clip")
    ]

    if len(recommended) > 0:

        best = recommended.sort_values(
            "Final Training Loss"
        ).iloc[0]

        print("\nRecommended Setting:")
        print(best.to_string())

print("\nExperiment Completed Successfully!")
```

OUTPUT

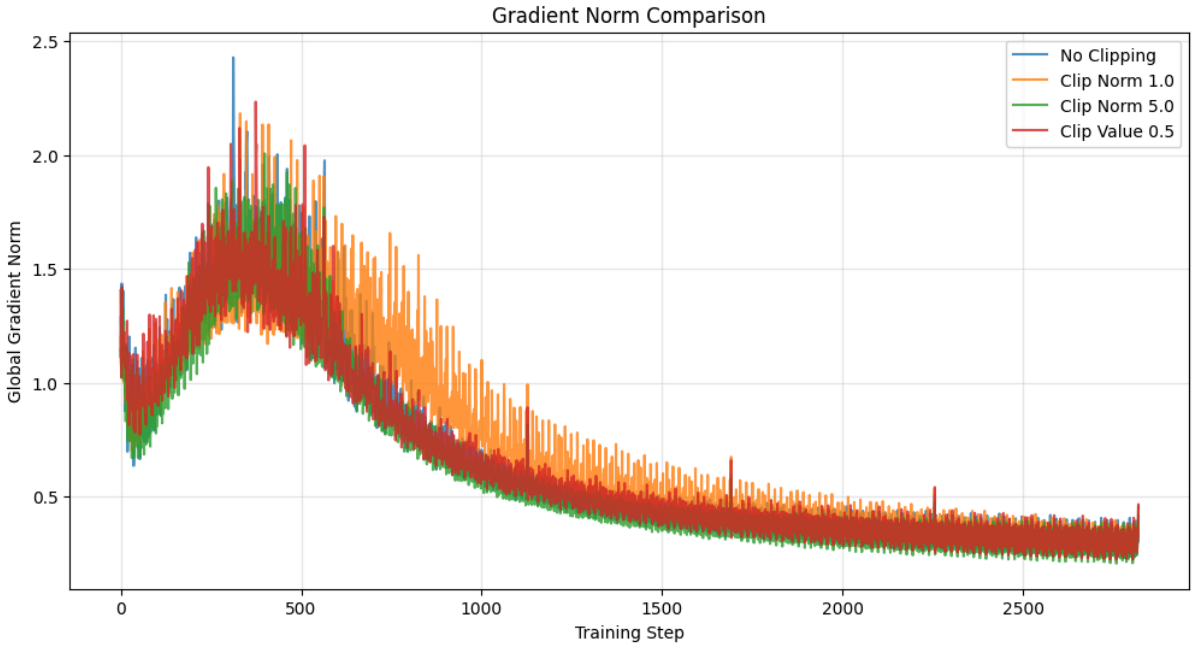
```
Total Characters: 36100
Vocabulary Size: 33
Input Shape: (36060, 40, 33)
Target Shape: (36060, 40)

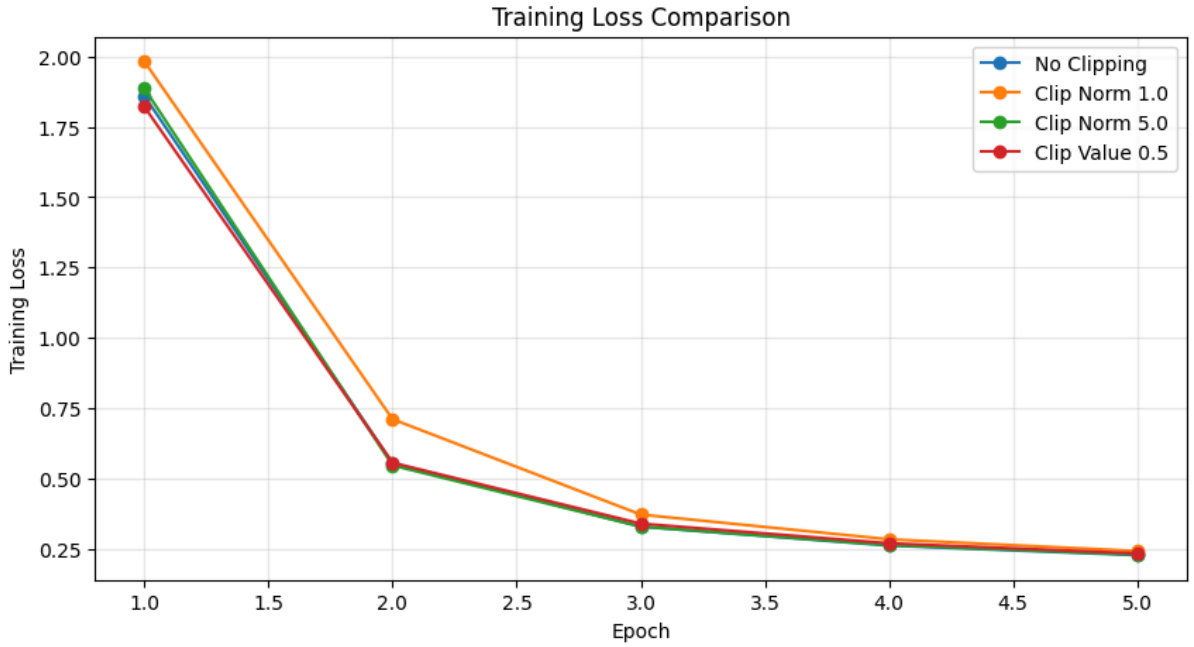
Training: No Clipping
Epoch 1/5 - Loss: 1.8597
Epoch 2/5 - Loss: 0.5558
Epoch 3/5 - Loss: 0.3307
Epoch 4/5 - Loss: 0.2635
Epoch 5/5 - Loss: 0.2295

Training: Clip Norm 1.0
Epoch 1/5 - Loss: 1.9817
Epoch 2/5 - Loss: 0.7121
Epoch 3/5 - Loss: 0.3735
Epoch 4/5 - Loss: 0.2858
Epoch 5/5 - Loss: 0.2441

Training: Clip Norm 5.0
Epoch 1/5 - Loss: 1.8863
Epoch 2/5 - Loss: 0.5480
Epoch 3/5 - Loss: 0.3308
Epoch 4/5 - Loss: 0.2655
Epoch 5/5 - Loss: 0.2316

Training: Clip Value 0.5
Epoch 1/5 - Loss: 1.8211
Epoch 2/5 - Loss: 0.5575
Epoch 3/5 - Loss: 0.3415
Epoch 4/5 - Loss: 0.2718
Epoch 5/5 - Loss: 0.2357
```


Gradient Norm Comparison
No Clipping
Clip Norm 1.0
Clip Norm 5.0
Clip Value 0.5
Global Gradient Norm
Training Step
2.5
2.0
1.5
1.0
0.5
0
500
1000
1500
2000
2500


Training Loss Comparison
No Clipping
Clip Norm 1.0
Clip Norm 5.0
Clip Value 0.5
Training Loss
Epoch
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
5.0

```
 No Clipping
Gradient Spikes: 0
Non-finite Loss/Gradient Batches: 0

 Clip Norm 1.0
Gradient Spikes: 0
Non-finite Loss/Gradient Batches: 0

 Clip Norm 5.0
Gradient Spikes: 0
Non-finite Loss/Gradient Batches: 0

 Clip Value 0.5
Gradient Spikes: 0
Non-finite Loss/Gradient Batches: 0

GRADIENT CLIPPING COMPARISON
        Method  Average Gradient Norm  Maximum Gradient Norm  Final Training Loss  Non-finite Batches
   No Clipping                 0.6681                 2.4272               0.2295                   0
 Clip Norm 1.0                 0.7114                 2.1819               0.2441                   0
 Clip Norm 5.0                 0.6261                 2.0060               0.2316                   0
Clip Value 0.5                 0.6610                 2.2327               0.2357                   0

Recommended Setting:
Method                   Clip Norm 5.0
Average Gradient Norm         0.626066
Maximum Gradient Norm         2.005962
Final Training Loss           0.231572
Non-finite Batches                   0

Experiment Completed Successfully!
```

5. RESULT

A character-level SimpleRNN was implemented using TensorFlow and Keras to study exploding gradients during text-generation training. Global gradient norms were calculated using `tf.GradientTape`, and gradient clipping was applied using norm-based and value-based methods with different thresholds. Gradient norm and training loss graphs were plotted to compare the different settings. The experiment demonstrates how gradient clipping helps control excessively large gradients, improve training stability, and reduce the risk of numerical problems during RNN training.